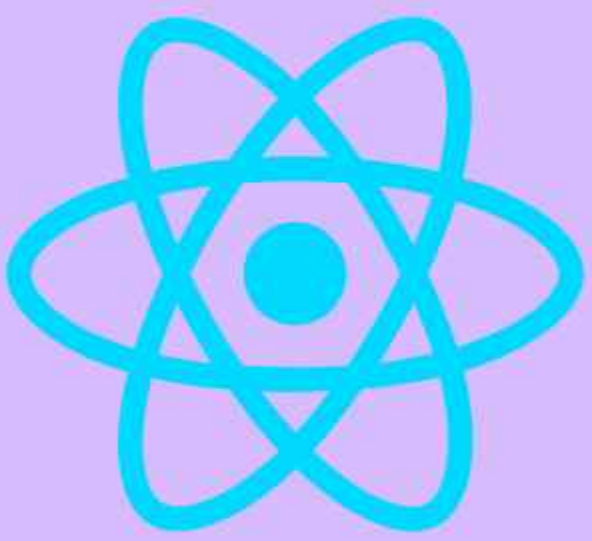
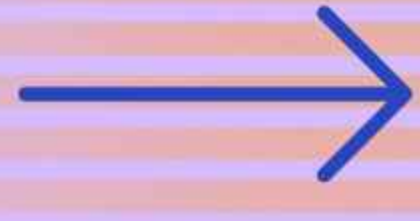
React and GraphQL

Fetch data efficiently



Vladyslav Demirov
@vladyslav-demirov

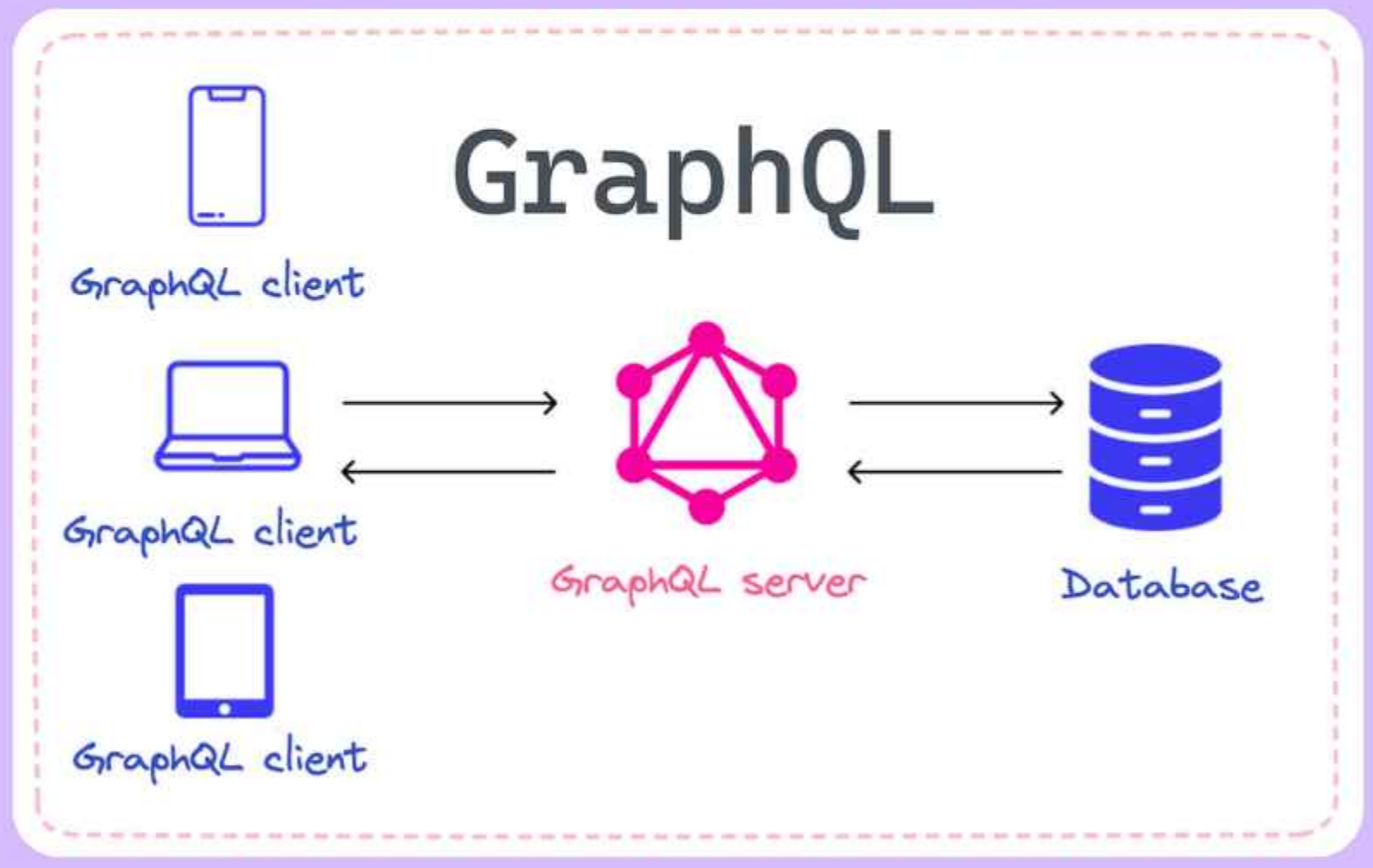


What is GraphQL?

💡 **GraphQL** is a query language for APIs that:

- Allows clients to request only the data they need.
- Provides a single endpoint for all data operations (queries, mutations, subscriptions).
- Enables real-time updates with subscriptions.

⚡ **Key Use Case**: Building flexible, efficient, and scalable data-driven apps.



Vladyslav Demirov

Use GraphQL with React?

📌 Step 1: Install Apollo Client

bash

```bash
npm install @apollo/client graphql
```

📌 Step 2: Initialize Apollo Client

jsx

```jsx
import { ApolloClient, InMemoryCache, ApolloProvider } from '@apollo/client';

const client = new ApolloClient({
  uri: 'https://your-graphql-endpoint.com/graphql',
  cache: new InMemoryCache(),
});

function App() {
  return (
    <ApolloProvider client={client}>
      <YourAppComponent />
    </ApolloProvider>
  );
}
```

Vladyslav Demirov

Writing GraphQL Query

📌 Step 3: Define a Query

jsx

```jsx
import { gql } from '@apollo/client';

const GET_DATA = gql`
  query GetData {
    users {
      id
      name
      email
    }
  }
`;
```

📌 Step 4: Use the Query in a Component

jsx

```jsx
import { useQuery } from '@apollo/client';

function UsersList() {
  const { loading, error, data } = useQuery(GET_DATA);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.users.map((user) => (
        <li key={user.id}>{user.name} - {user.email}</li>
      ))}
    </ul>
  );
}
```

Vladyslav Demirov

Real-Time Updates with Subscriptions

📌 Step 5: Add Subscriptions

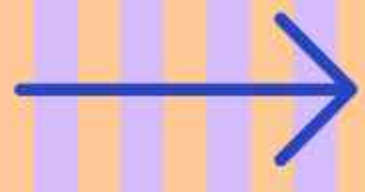
jsx

```jsx
import { useSubscription } from '@apollo/client';

const NEW_USER_SUBSCRIPTION = gql`
  subscription OnUserAdded {
    userAdded {
      id
      name
      email
    }
  }
`;

function NewUserAlert() {
  const { data } = useSubscription(NEW_USER_SUBSCRIPTION);
  return data ? <p>New user: {data.userAdded.name}</p> : null;
}
```

Vladyslav Demirov



Key Benefits of GraphQL

1 Efficient Data Fetching

- Fetch only the data you need, reducing payload size.

2 Single Endpoint

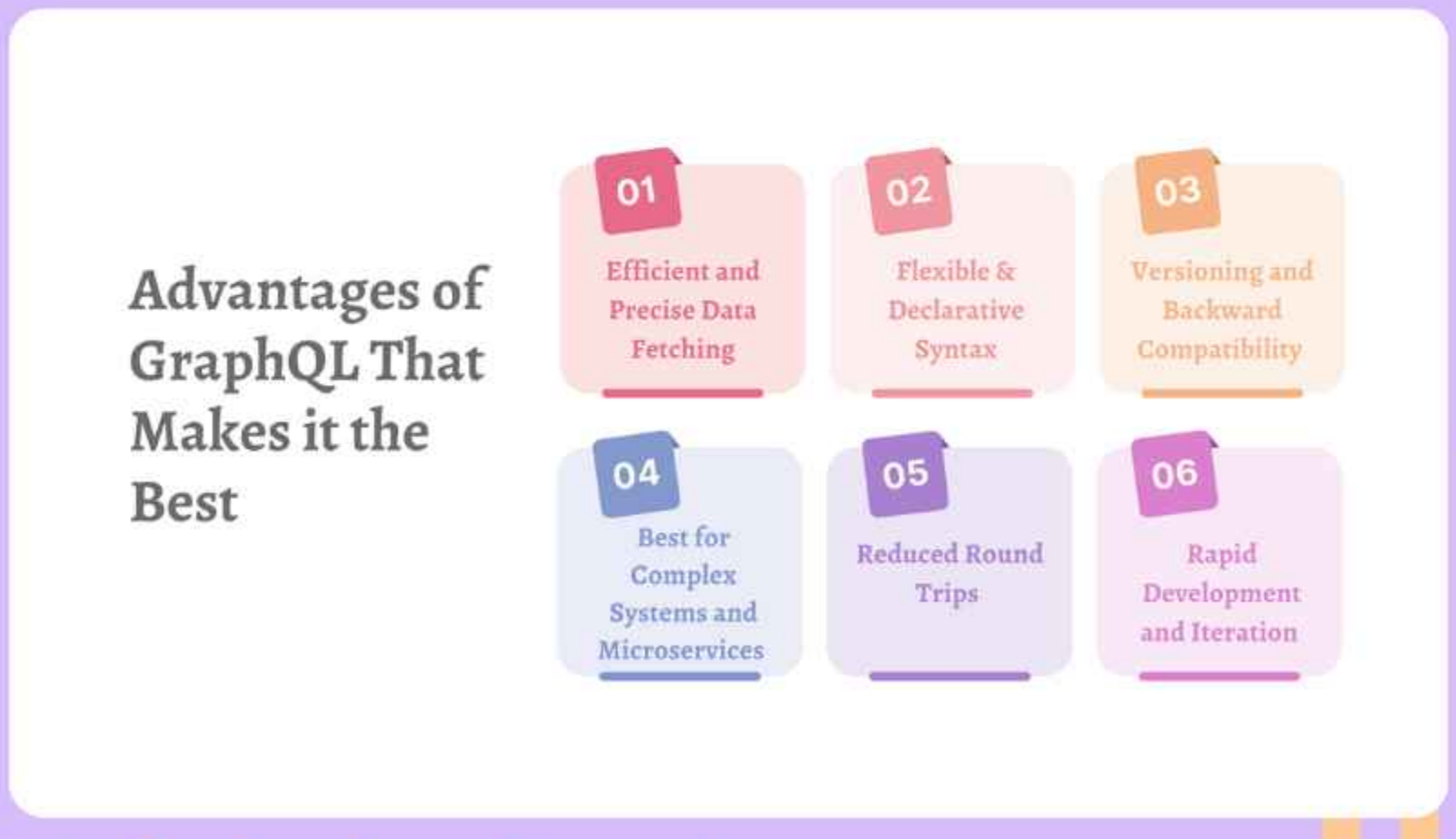
- Simplify API management with one endpoint for all operations.

3 Real-Time Updates

- Use subscriptions for live data updates (e.g., chat apps, live dashboards).

4 Strongly Typed Schema

- Define clear data structures and avoid runtime errors.



Vladyslav Demirov

Real-World Use Cases

1 Social Media Apps

- Fetch user profiles, posts, and comments in a single query.

2 E-Commerce Platforms

- Retrieve product details, reviews, and recommendations dynamically.

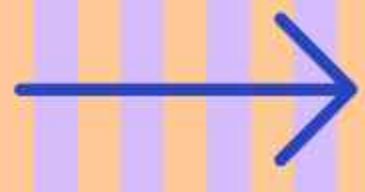
3 Dashboards

- Build real-time dashboards with live data updates.

4 Collaborative Tools

- Enable real-time collaboration (e.g., Google Docs, Trello).

Vladyslav Demirov



Conclusion

✅ Use Apollo Client for state management and caching.

✅ Optimize queries to avoid over-fetching.

✅ Leverage fragments for reusable query structures.

✅ Monitor performance with tools like Apollo Studio.

Vladyslav Demirov

HAPPY CODING

Vladyslav Demirov
@vladyslav-demirov